

Práctica 2

Modelo de memoria compartida

Pautas:

Compilar aplicación Pthreads en Linux gcc:

```
gcc -pthread -o <ejecutable> <fuente>
```

Compilar aplicación OpenMP en Linux gcc:

```
gcc -fopenmp -o <ejecutable> <fuente>
```

Se deben respetar las estructuras de programas y conceptos vistos durante la clase teórica. Específicamente para Pthreads:

Variables compartidas no se pasan como parámetros a los hilos.

Los hilos sólo reciben como parámetro su identificador.

No retornar valores usando funciones `pthread_exit` y `pthread_join`.

En soluciones por etapas no por realizar múltiples creaciones de hilos.

Sincronizar etapas por barreras.

Primero, proponga soluciones a partir de la estrategia de diseño de cuatro pasos de Foster, asumiendo que se ejecutará sobre una arquitectura multicore simétrica con P unidades de procesamiento, resuelva – con Pthreads y OpenMP - y analice cada problema por su cuenta, utilizando su propio razonamiento y conocimientos. Posteriormente, utilice dos o más motores de inteligencia artificial para profundizar en el tema, comparar respuestas analizando similitudes y diferencias entre estos motores de IA.

Ejecutar las soluciones con un número de hilos potencia de 2.

Validar los resultados de las soluciones paralelas respecto a los resultados de la solución secuencial. Ambos deben ser iguales.

Analice las ventajas y desventajas de implementación de cada herramienta.

Analice que posibilidades ofrecen otros lenguajes de programación de forma nativa, interpretados y compilados, como alternativa al lenguaje C para resolver cada problema.

1. Diseñar e implementar una solución paralela para cada programa de la práctica 1:
 - a. **mm_naive.c** optimizado.
 - b. **C=A.A**
 - c. **mmblok.c** y **mmblok_blas.c**, a partir del bloque que mejor rendimiento obtuvo en la solución secuencial.
2. Diseñar e implementar una solución paralela para calcular el valor mínimo, máximo y promedio de las matrices **AN.m** (**N** dimensión de la matriz) utilizadas en la práctica 1.
3. Analizar el siguiente fragmento de código OpenMP que traspone una matriz cuadrada de **NxN**:

```
#pragma omp parallel default(none) private(i,j,temp,timetick,tid) shared(A,N)
{
    tid= omp_get_thread_num();
    timetick = dwalltime();
    #pragma omp for private(i,j,temp) nowait
    for(i=0;i<N;i++){
        for(j=i+1;j<N;j++){
            temp = A[i*N+j];
            A[i*N+j]= A[j*N+i];
            A[j*N+i]= temp;
        }
    }
    printf("Tiempo para el thread %d: %f segs\n", tid,dwalltime() - timetick);
}
```

- a. Diseñar e implementar una solución paralela OpenMP completa para aplicarlo a las matrices **AN.m** de la práctica 1.
 - b. Aunque la solución es correcta, los tiempos de ejecución arrojados tiene un problema, describir de que problema se trata. ¿Qué cláusula usaría para corregirlo? Aplique la solución, ejecute de nuevo y compare con los resultados anteriores.
 - c. Diseñe e implemente una solución paralela en Pthreads ¿qué inconvenientes encuentra?
4. Diseñe una solución paralela para el algoritmo **nreinas.c** de la práctica 1.
 - a. ¿Qué estrategia de descomposición es la mas adecuada para este problema?
 - b. ¿Qué es una tarea? ¿Són las tareas resultantes estáticas o dinámicas? ¿Qué problema puede surgir si se aplica un mapeo estático?
 - c. Analice cuál de las dos herramientas es mas adecuada, Pthreads, OpenMP, o si es indistinto utilizar una u otra.
 - d. Utilizar la IA para desarrollar una solución funcional.

5. Diseñe una solución paralela para el algoritmo **mergeSimpleMalloc.c** de la práctica 1.
 - a. Analice cuál de las dos herramientas es mas adecuada, Pthreads, OpenMP, o si es indistinto utilizar una u otra.
 - b. Cuando se trata de algoritmos paralelos de ordenación ¿Por qué se prefiere *merge sort* a *quick sort*?
 - c. Utilizar la IA para desarrollar una solución funcional.